8. QUEUE USING ARRAY

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

int queue[MAX_SIZE];
int front = -1;
int rear = -1;

void enqueue(int item) {
    if (rear == MAX_SIZE - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1 && rear == -1) {
        front = 0;
        rear = 0;
    } else {
        rear++;
    }
    queue[rear] = item;
    printf("%d enqueued to queue\n", item);
}

int dequeue() {
    int item;
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return -1;
    }
    item = queue[front];
    front++;

    // Reset queue if it's empty now
    if (front > rear) {
        front = -1;
        rear = -1;
    }
    return item;
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
```

```c
    int choice, item;
    while (1) {
        printf("\nQueue Operations:\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter item to enqueue: ");
                scanf("%d", &item);
                enqueue(item);
                break;
            case 2:
                item = dequeue();
                if (item != -1) {
                    printf("%d dequeued from queue\n", item);
                }
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

PROGRAM OUTPUT